Reinforcement Learning: From Interaction to Temporal-Difference Learning

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

June 1, 2026

Primary Reference and Scope

Primary source. Sutton and Barto, *Reinforcement Learning: An Introduction*, 2nd edition draft.

- Chapter 1: the reinforcement learning problem and the role of value.
- Chapter 3: finite Markov decision processes, returns, policies, and Bellman equations.
- Chapter 6: temporal-difference learning, Sarsa, and Q-learning.
- Chapter 7: n-step methods, TD(λ), and eligibility traces.
- Supporting bridge: dynamic programming, Monte Carlo, and generalized policy iteration.

A One-Sentence Main Line

Reinforcement learning is the study of how an agent improves a policy from interaction by estimating long-term value and using those estimates to change future behavior.

Lecture Roadmap

1. Start with the agent–environment loop.
2. Formalize the loop as a finite MDP.
3. Define the objective through return and value functions.
4. Use Bellman equations to connect current value with future value.
5. View algorithms as different ways to do backups.
6. Move from Monte Carlo and dynamic programming to TD learning.
7. Extend one-step TD to multi-step credit assignment through eligibility traces.

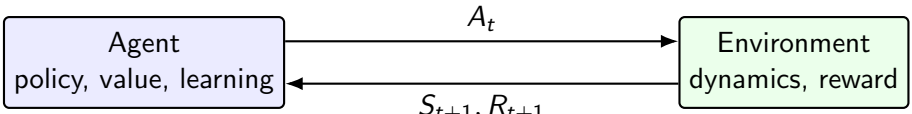
Outline

What Makes RL Different?

Setting	Supervised Learning	Reinforcement Learning
Data	Labeled examples (x, y)	Experience stream $(S_t, A_t, R_{t+1}, S_{t+1})$
Feedback	Correct target is visible	Reward is scalar, delayed, and partial
Decision effect	Prediction does not change data source	Actions change future states and data
Goal	Minimize prediction loss	Maximize long-term return

Core difficulty: the agent must learn from consequences of its own behavior.

Agent–Environment Interface



At each time step: observe state, choose action, receive reward and next state.

Four Elements of an RL Agent

- Policy π: the agent's behavior rule, often $\pi(a \mid s)$.
- Reward signal R_{t+1}: immediate feedback that defines the task.
- Value function v_π, q_π: prediction of long-term reward.
- Model $p(s', r \mid s, a)$: optional prediction of environment response.

The key distinction

Reward says what is good immediately. Value estimates what is good after future consequences are considered.

Reward Is Not the Same as Value

Immediate reward

- A scalar signal after one transition.
- Defines the objective externally.
- May be sparse or misleading locally.

Value

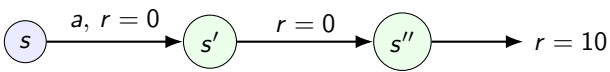
- A prediction about future cumulative reward.
- Used to compare states or actions.
- Learned from experience and bootstrapping.

$$\text{good action} \neq \arg\max_a \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

$$\text{good action} = \arg\max_a \mathbb{E}[G_t \mid S_t = s, A_t = a].$$

Delayed Consequences

- An action can lower immediate reward but lead to better future states.
- An action can give high immediate reward but destroy future options.
- Learning must assign credit or blame across time.



The action at s matters because it changes the distribution of future rewards.

Exploration Versus Exploitation

- Exploitation: choose actions that look best under current estimates.
- Exploration: choose actions to improve future estimates.
- RL requires both because the data distribution is controlled by the policy.

$$\text{Let } a^*(s) \in \arg\max_b Q(s,b), \qquad \epsilon\text{-greedy:} \quad \pi(a \mid s) = \begin{cases} 1-\epsilon+\epsilon/|\mathcal{A}(s)|, & a = a^*(s), \\ \epsilon/|\mathcal{A}(s)|, & \text{otherwise.} \end{cases}$$

This simple rule is often enough to expose the exploration issue clearly.

Prediction and Control

Prediction

- Policy π is fixed.
- Learn v_π or q_π.
- Question: how good is this behavior?

Control

- Policy changes during learning.
- Learn values and improve policy.
- Question: how should the agent behave?

Main algorithmic pattern

Most tabular control algorithms repeatedly alternate between value estimation and policy improvement.

Model-Based and Model-Free

Approach	What is learned or known	How actions improve
Model-based	Estimate or use $p(s', r \mid s, a)$	Plan by simulating backups
Model-free	Estimate v_π or q_π directly	Improve policy from value estimates
Policy search	Parameterize π_θ directly	Optimize expected return

Sutton–Barto Chapters 6 and 7 focus on model-free value learning in the tabular setting.

Tic-Tac-Toe as the First Value-Learning Picture

- Keep a table with one number for each board position.
- Interpret the number as the current estimate of eventual win probability.
- Pick moves that lead to high-value positions, with occasional exploration.
- After the game, adjust earlier values toward later values and outcomes.

$$V(S_t) \leftarrow V(S_t) + \alpha\big[V(S_{t+1}) - V(S_t)\big]$$

This is the seed of the backup idea: later estimates teach earlier estimates.

Outline

Finite MDP

A finite Markov decision process is specified by

$$(\mathcal{S}, \mathcal{A}, p, \gamma), \qquad p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

- $\mathcal{S}$: finite state set.
- $\mathcal{A}(s)$: finite action set available in state s.
- p: one-step transition and reward law.
- $\gamma \in [0, 1]$: discount factor.

Once p is known and the state is Markov, all future prediction can be built from one-step dynamics.

The Markov Property

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_0, A_0, R_1, \ldots, S_t, A_t) = p(s', r \mid S_t, A_t).$$

- The current state contains all history that is useful for predicting the next step.
- Markov does not mean the state contains every physical fact.
- It means no forgotten history would improve the conditional distribution relevant to the task.

Practical reading

In real systems, the state representation is often only approximately Markov; the theory tells us what information the representation should try to preserve.

Dynamics Decompositions

From the joint one-step law,

$$p(s', r \mid s, a),$$

we often use derived quantities:

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a),$$

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s', r} r \, p(s', r \mid s, a),$$

$$r(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'].$$

The joint form is the cleanest because reward and next state can be statistically coupled.

Policies

A policy defines behavior:

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

- Deterministic policy: $\pi(s) \in \mathcal{A}(s)$.
- Stochastic policy: distribution over actions.
- Stationary policy: same mapping at every time step.

Given an MDP and a policy π, the interaction becomes a Markov reward process. Prediction is then well-defined.

Return: The Quantity We Optimize

The return from time t is the discounted sum of future rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

- Episodic tasks can terminate; terminal states produce no further reward.
- Continuing tasks often require $\gamma < 1$ to make the infinite sum finite.
- $\gamma = 0$ is myopic; $\gamma \to 1$ is farsighted.

Episodic and Continuing Tasks

Episodic

- Interaction breaks into episodes.
- Examples: games, trips through a maze.
- Return can be finite even with $\gamma = 1$.

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

Continuing

- No natural terminal time.
- Examples: process control, long-lived robot.
- Discounting is the standard finite-return device.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Reward Design: The Objective Boundary

- The reward signal defines the task, not the algorithm.
- It should represent what we want the agent to achieve, not how we want it to achieve it.
- Bad reward design creates policies that optimize the signal while missing the intent.

Useful rule

Reward is the external objective. Value is the agent's learned prediction of that objective under a policy.

State-Value Function

For a policy π, the value of state s is

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

- It answers: if we start here and follow π, how much return should we expect?
- It is a prediction problem once π is fixed.
- It is the central object behind policy evaluation.

Action-Value Function

For a policy π, the value of taking action a in state s is

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

- It answers: what if we force the first action to be a, then follow π?
- It is more directly useful for control when the model is unknown.
- A greedy policy can be read directly from q:

$$\pi_{\text{greedy}}(s) \in \arg\max_a q(s, a).$$

Bellman Expectation Equation for v_π

Split return into one reward plus discounted future return:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Therefore

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big].$$

Interpretation

Value is self-consistent: it equals expected immediate reward plus discounted value of the next state.

Bellman Expectation Equation for q_π

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s',a') \right].$$

- The first action is fixed to a.
- At the next state, actions follow π.
- This equation underlies Sarsa.

Optimal Value Functions

$$v_*(s) = \max_\pi v_\pi(s), \qquad q_*(s,a) = \max_\pi q_\pi(s,a).$$

- v_* tells us the best achievable return from a state.
- q_* tells us the best achievable return after a specified first action.
- Any policy greedy with respect to q_* is optimal:

$$\pi_*(s) \in \arg\max_a q_*(s,a).$$

Bellman Optimality Equations

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_*(s')\big],$$

$$q_*(s, a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \max_{a'} q_*(s', a')\big].$$

Control as fixed-point search

Finding an optimal policy can be reduced to finding the fixed point of an optimal Bellman backup.

Bellman Equation as a Fixed-Point Problem

For a fixed policy π, the Bellman expectation equation is a linear system:

$$\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma P_\pi \mathbf{v}_\pi, \qquad (I - \gamma P_\pi)\mathbf{v}_\pi = \mathbf{r}_\pi.$$

$$\mathbf{v}_\pi = (I - \gamma P_\pi)^{-1}\mathbf{r}_\pi \quad \text{if the inverse is feasible to compute.}$$

For control, the optimal Bellman equation is nonlinear:

$$v_* = T_* v_*, \qquad (T_* V)(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big].$$

Policy iteration and value iteration are practical ways to solve these Bellman fixed-point equations.

Backup Diagrams Without the Diagrams

Almost every tabular RL algorithm can be understood by three choices:

1. What target? full return, one-step bootstrapped target, or multi-step mixture.
2. What expectation? exact expectation over p, sample transition, or sampled episode.
3. What policy? evaluate current policy, behave with another policy, or improve greedily.

$$\text{old estimate} \leftarrow \text{old estimate} + \alpha(\text{target} - \text{old estimate}).$$

Outline

Policy Evaluation

Given policy π, estimate v_π.

Dynamic programming update when p is known:

$$V_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V_k(s')\big].$$

This is a full-width Bellman expectation backup: average over all actions, next states, and rewards.

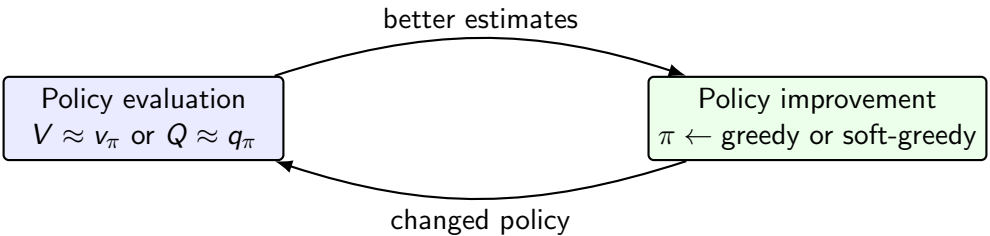
Policy Improvement

Given v_π, choose actions greedily with respect to one-step lookahead:

$$\pi'(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big].$$

- If π' is greedy with respect to v_π, then it is no worse than π.
- Repeating evaluation and improvement leads to policy iteration.

Generalized Policy Iteration



GPI does not require exact evaluation before improvement.
Most practical RL algorithms interleave both processes.

How Evaluation and Improvement Alternate

Exact policy iteration

1. Fix π_k.
2. Evaluate until $V \approx v_{\pi_k}$ is accurate.
3. Improve once:

$$\pi_{k+1}(s) \in \arg\max_a q_{\pi_k}(s, a).$$

Generalized policy iteration

1. Do a few value updates.
2. Improve the policy a little.
3. Repeat without waiting for full convergence.

Main point

Policy evaluation asks "how good is the current policy?" Policy improvement asks "how should the policy change given these values?"

Solving Bellman Equations: Policy and Value Iteration

Policy iteration solves

$$v_{\pi_k} = T_{\pi_k} v_{\pi_k}$$

1. Evaluate v_{π_k}.
2. Improve π_k greedily.
3. Stop when policy is stable.

Value iteration solves

$$v_* = T_* v_*$$

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V_k(s')\big].$$

Evaluation and improvement are merged into one optimality backup.

These methods are exact dynamic programming solvers when p is known.

Monte Carlo Evaluation

If the model is unknown but complete episodes are available:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[G_t - V(S_t)\big].$$

- No bootstrapping: the target is an observed return.
- No model: uses sampled experience.
- Naturally episodic: must wait until enough future reward is observed.

Monte Carlo is simple, unbiased for the value under the sampled policy, and can have high variance.

DP, MC, and TD: The Three Backup Styles

Method	Target	Model needed?	Wait for episode end?
Dynamic programming	Expected Bellman target	Yes	No
Monte Carlo	Sampled full return G_t	No	Usually yes
Temporal difference	Sampled bootstrapped target	No	No

$$\text{TD target} = R_{t+1} + \gamma V(S_{t+1}).$$

Sampling and Bootstrapping

Sampling

- Use experienced transitions.
- Avoid summing over all next states.
- Introduces sampling noise.

Bootstrapping

- Use current value estimate in the target.
- Learn before final outcome is known.
- Introduces bias from current estimates.

TD learning is the central combination: sampled transitions plus bootstrapped targets.

Outline

TD Prediction

TD(0) updates after each transition:

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t,$$

where

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

- δ_t is the temporal-difference error.
- It measures surprise relative to the current one-step prediction.
- It is also the learning signal used in many actor–critic algorithms.

TD(0) Algorithm

Input: policy π, step size α, initial V.

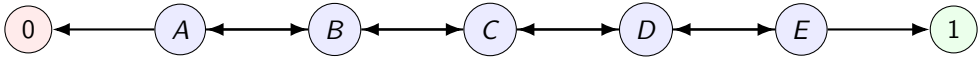
1. Initialize S at the start of an episode.
2. Choose $A \sim \pi(\cdot \mid S)$.
3. Take action A, observe R and S'.
4. Update
$$V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big].$$
5. Set $S \leftarrow S'$ and continue until termination.

For terminal S', use $V(S') = 0$.

Why TD Is Between MC and DP

Property	Monte Carlo	TD(0)
Target	G_t	$R_{t+1} + \gamma V(S_{t+1})$
Uses samples?	Yes	Yes
Bootstraps?	No	Yes
Online update?	Only after return known	Immediately
Variance	Higher	Lower
Bias from estimate	Lower	Higher

Random Walk Example



- Start in C; move left or right with equal probability.
- Reward is 1 only when terminating on the right.
- True values are probabilities of eventual right termination.

TD updates intermediate states before the final terminal reward has been fully propagated by Monte Carlo averaging.

Batch TD Intuition

With a fixed batch of experience:

- Monte Carlo estimates values that fit the observed returns in the batch.
- TD(0) estimates values that are consistent with the maximum-likelihood Markov model induced by the batch.

Why this matters

If the Markov assumption is right, TD can generalize across repeated transitions: a transition from A to B teaches A through the current estimate of B.

From State Values to Action Values

For control without a model, it is convenient to learn $Q(s, a)$.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(\text{target} - Q(S_t, A_t)).$$

- If the target follows the actually selected next action, we get Sarsa.
- If the target uses the greedy next action, we get Q-learning.
- The difference is the difference between on-policy and off-policy control.

Sarsa: On-Policy TD Control

The update uses the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right].$$

- Behavior policy and target policy are the same.
- Usually use ϵ-greedy policy derived from Q.
- Learns the value of the exploratory policy it actually follows.

Sarsa Algorithm

1. Initialize $Q(s, a)$.
2. For each episode, initialize S and choose A from the current policy.
3. Take A, observe R, S'.
4. Choose A' from the current policy at S'.
5. Update
$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)].$$
6. Set $S \leftarrow S'$, $A \leftarrow A'$ and repeat.

Q-Learning: Off-Policy TD Control

The update uses the greedy value of the next state:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

- Behavior policy may be exploratory.
- Target policy is greedy with respect to Q.
- Learns an approximation to q_* while behaving non-greedily.

Q-Learning Algorithm

1. Initialize $Q(s, a)$.
2. For each episode, initialize S.
3. Choose A from a behavior policy, e.g. ϵ-greedy from Q.
4. Take A, observe R, S'.
5. Update
$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)].$$
6. Set $S \leftarrow S'$ and repeat.

Sarsa Versus Q-Learning

Question	Sarsa	Q-learning
Policy type	On-policy	Off-policy
Target	$R + \gamma Q(S', A')$	$R + \gamma \max_a Q(S', a)$
Learns value of	Exploratory behavior policy	Greedy target policy
Risk-sensitive effect	Accounts for exploratory actions	Ignores future exploration in target

Cliff-Walking Intuition

- The shortest path near a cliff is optimal for a perfectly greedy policy.
- With ϵ-greedy exploration, the agent sometimes slips into bad actions.
- Sarsa evaluates the policy including these exploratory slips.
- Q-learning evaluates the greedy policy while still using exploratory behavior.

Takeaway

On-policy methods optimize what the agent actually does; off-policy methods can learn about a different target policy from exploratory data.

Expected Sarsa as a Useful Middle Point

Expected Sarsa replaces the sampled next action by its expectation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

- Same target policy idea as Sarsa.
- Lower variance than sampling A_{t+1}.
- Becomes Q-learning when π is greedy in the target.

Convergence Conditions in the Tabular Case

For one-step tabular control, convergence statements usually require:

- finite state and action spaces;
- sufficient exploration: all relevant state–action pairs visited infinitely often;
- decreasing step sizes such as

$$\sum_t \alpha_t(s,a) = \infty, \qquad \sum_t \alpha_t(s,a)^2 < \infty;$$

- for on-policy control, policies become greedy in the limit while still exploring enough.

Constant step sizes are common in nonstationary or approximate settings, but exact convergence guarantees change.

Outline

Why One-Step TD Is Not Enough

One-step TD assigns each reward through one transition at a time:

$$R_{t+1} + \gamma V(S_{t+1}).$$

- It is online and low variance.
- But reward information may propagate slowly over long horizons.
- Monte Carlo uses the full outcome but waits and has higher variance.

Chapter 7 introduces a spectrum between these extremes.

n-Step Return

The n-step return bootstraps after n rewards:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

$$V(S_t) \leftarrow V(S_t) + \alpha \big[G_{t:t+n} - V(S_t) \big].$$

- $n = 1$: TD(0).
- n reaches terminal time: Monte Carlo target.

Choosing n

Small n

- More bootstrapping.
- Lower variance.
- Faster online updates.
- More bias from current estimates.

Large n

- Less bootstrapping.
- More real reward information.
- Higher variance.
- Slower credit assignment online.

TD(λ) avoids choosing just one n by averaging many n-step returns.

Forward View of TD(λ)

The λ-return is an exponentially weighted average of n-step returns:

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}.$$

For episodic tasks with terminal time T:

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_t.$$

Then update toward G_t^λ:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[G_t^\lambda - V(S_t)\big].$$

Endpoints of λ

- If $\lambda = 0$:

$$G_t^\lambda = G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1}),$$

  so TD(λ) becomes TD(0).

- If $\lambda = 1$ in an episodic task:

$$G_t^\lambda = G_t,$$

  so the target becomes the Monte Carlo return.

λ controls the degree of bootstrapping.

Forward View Is Conceptual, Not Causal

The forward view asks:

For the state visited at time t, what sequence of future rewards and future estimates should determine its update?

- It explains the target.
- But it needs future rewards before updating S_t.
- The backward view gives a causal implementation using eligibility traces.

Eligibility Trace

For each state s, maintain a trace $E_t(s)$ measuring recent visitation.

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}\{S_t = s\}.$$

- Recently visited states have large traces.
- Traces decay by $\gamma\lambda$.
- The current TD error updates all states in proportion to their traces.

Backward View of TD(λ)

Compute the one-step TD error:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

Update every state:

$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s), \qquad \forall s \in \mathcal{S}.$$

Mechanism

When a surprising reward or transition occurs now, the trace sends credit backward to states visited recently.

TD(λ) Algorithm

1. Initialize $V(s)$.
2. At the start of each episode, set $E(s) = 0$ for all s.
3. At each step, observe S, A, R, S' under policy π.
4. Compute $\delta = R + \gamma V(S') - V(S)$.
5. Increase the trace of the visited state: $E(S) \leftarrow E(S) + 1$.
6. For all s:

$$V(s) \leftarrow V(s) + \alpha\delta E(s), \qquad E(s) \leftarrow \gamma\lambda E(s).$$

Accumulating and Replacing Traces

Accumulating trace

$$E(S_t) \leftarrow E(S_t) + 1.$$

- Multiple close visits accumulate.
- Natural from the classical derivation.

Replacing trace

$$E(S_t) \leftarrow 1.$$

- Caps the active state's trace.
- Often useful in tabular tasks with loops.

Both express the same idea: eligibility is a fading memory of recent responsibility.

Sarsa(λ)

Use traces over state–action pairs:

$$E_t(s,a) = \gamma\lambda E_{t-1}(s,a) + \mathbf{1}\{S_t = s, A_t = a\}.$$

The TD error is the Sarsa error:

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t).$$

Update all pairs:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\delta_t E_t(s,a).$$

Sarsa(λ) Algorithm

1. Initialize $Q(s, a)$.
2. At episode start, set all $E(s, a) = 0$.
3. Choose A from the current behavior policy.
4. Take A, observe R, S', choose A'.
5. Compute $\delta = R + \gamma Q(S', A') - Q(S, A)$.
6. Set $E(S, A) \leftarrow E(S, A) + 1$.
7. For all (s, a):

$$Q(s, a) \leftarrow Q(s, a) + \alpha\delta E(s, a), \qquad E(s, a) \leftarrow \gamma\lambda E(s, a).$$

Watkins's Q(λ)

Q-learning can also be combined with traces:

$$\delta_t = R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t).$$

- If the next action is greedy, traces continue.
- If the next action is exploratory, traces are cut off.
- Reason: off-policy backups should not credit earlier actions through a later action that deviates from the greedy target policy.

This is an early example of the extra care needed for off-policy multi-step learning.

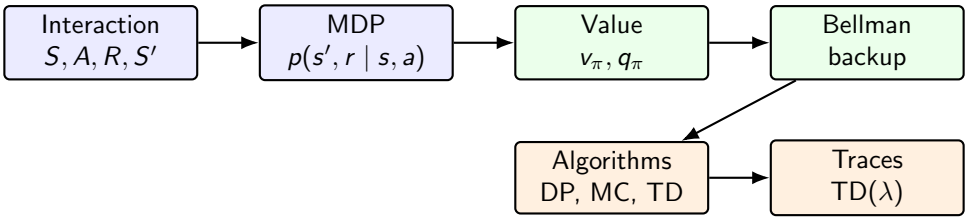
How to Read λ Practically

- $\lambda = 0$: stable, myopic credit assignment, low variance.
- Intermediate λ: often best empirical tradeoff.
- $\lambda \approx 1$: long credit assignment, closer to Monte Carlo, higher variance.

Mental model

$\gamma\lambda$ is the decay rate of memory. A past state remains eligible only while its trace has not faded away.

The Full Tabular Main Line



Everything is organized around one idea: estimate long-term value from experience, then use value to improve behavior.

Outline

What Changes with Function Approximation?

Tabular methods store one number per state or state–action pair.

$$V(s) \quad \longrightarrow \quad \hat{v}(s; w), \qquad Q(s, a) \quad \longrightarrow \quad \hat{q}(s, a; w).$$

- Generalization becomes possible across states.
- Stability becomes harder because updates couple many states.
- The deadly triad appears: function approximation, bootstrapping, and off-policy learning.

What Survives in Deep RL?

- The experience tuple $(S_t, A_t, R_{t+1}, S_{t+1})$.
- The return objective.
- Bellman targets and TD errors.
- On-policy versus off-policy distinctions.
- Multi-step returns and trace-like credit assignment.
- Generalized policy iteration: evaluate, improve, repeat.

DQN, actor–critic, PPO, and many RLHF variants still inherit this conceptual skeleton.

A Minimal Bridge to Policy Gradient

Value methods improve a policy through value estimates. Policy-gradient methods directly optimize policy parameters:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0],$$

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \widehat{A}_t\right].$$

- $\widehat{A}_t$ is often built from TD errors or multi-step returns.
- The value function returns as a critic or baseline.

LLM Post-Training Through the RL Lens

RL object	LLM post-training analogy
State S_t	Prompt plus generated context so far
Action A_t	Next token or higher-level response decision
Policy π_θ	Language model distribution
Reward R	Human preference model, rule-based score, task metric
Return G	Overall quality or task success of a response/trajectory
Value/critic	Learned prediction of future reward or advantage

The MDP abstraction is still useful, but state/action spaces are enormous and rewards are often learned proxies.

Common Failure Modes

- Reward misspecification: optimizing the signal instead of the intended goal.
- Poor exploration: never collecting data for better actions.
- High variance: long-horizon returns make learning noisy.
- Instability: bootstrapping plus approximation can diverge.
- Distribution shift: learned values are used on states unlike the data.

The tabular theory gives clean definitions; modern systems inherit the same issues at scale.

Outline

The Main Story in Five Equations

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')]$$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}\{S_t = s\}.$$

Algorithm Map

Algorithm family	Target	Main use
DP evaluation	Expected Bellman target	Known model prediction
Value iteration	Bellman optimality target	Known model control
Monte Carlo	Full sampled return	Episodic model-free prediction/control
TD(0)	One-step sampled Bellman target	Online model-free prediction
Sarsa	On-policy action-value TD target	Online exploratory control
Q-learning	Greedy off-policy TD target	Learning greedy target policy
TD(λ)	Mixture of n-step targets	Faster temporal credit assignment

Key Takeaways

- RL is about learning behavior from interaction, not just fitting labels.
- MDPs make the problem mathematically precise through state, action, transition, reward, and discount.
- Value functions transform delayed reward into a local decision score.
- Bellman equations are the fixed-point structure behind prediction and control.
- TD learning combines sampling and bootstrapping.
- Eligibility traces turn one-step TD into multi-step credit assignment.

Open Questions

If you are interested in these questions, please feel free to write an email to me (hpp@bimsa.cn) with your comments.

1. How should reward be specified when the true objective is hard to measure?
2. How can agents explore efficiently in enormous state and action spaces?
3. Which parts of tabular convergence intuition survive deep function approximation?
4. How should RL algorithms handle learned, nonstationary reward models?
5. What is the right abstraction level for RL in language-model agents?

Welcome Inputs

Any comments?

Welcome your inputs!